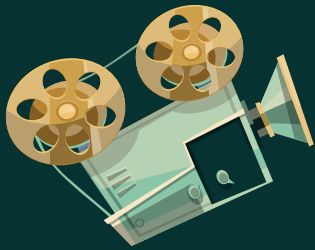


IMDb Sentiment Analysis



Disha Khati, Kailash Turimella, Tyler Soong

MOTIVATION

We are all avid Letterboxd users!!

What we wanted to explore: Can we use machine learning methods to read a review and know if it's positive or negative?

Broader applications: recommendation systems, brand monitoring, film industry analytics



QPDAC...

I.

Question?

Can TF-IDF and LDA features predict review sentiment?

II.

Our plan...

Clean text → extract features → train classifiers → compare

III.

Data

50k IMDB reviews,
balanced 25k positive /
25k negative

IV.

Analysis

7 models across 2
classifier families

Conclusion towards the end, we are not giving it away just yet ...

Text Cleaning pipeline

Raw reviews contained HTML tags (
), punctuation, numbers. So we had to do some cleaning...

Step 1: Stripping HTML with regex

Step 2: Removing punctuation and numbers

Step 3: Lowercase everything

Step 4: Removing stopwords ("the", "a", "is", words with no signal)

Step 5: *Lemmatize* — reducing words to base form (*lemma*);
"running" → "run" · "movies" → "movie" · "acting" → "act"



Text Cleaning in Action...

Review — POSITIVE

Before:

"Dressed to Kill" has been more or less forgotten in critical circles in the past 20 years, but it is a true American classic, a film which is much more than just a glossy thriller.

I sincerely hope the DVD release will give more people the chance to hear about it and see it.

After:

dressed kill less forgotten critical circle past year true american classic film much glossy thriller sincerely hope dvd release give people chance hear see



Review — NEGATIVE

Before:

The movie starts out fine. Widower out with new girlfriend and the children.

The movie is filled with stupid director's choices. Like "lets separate." "I am coming down to...." do what? Stupid Stupid Stupid.

Please do not waste your time hoping that it will get better..... Not hardly.

After:

movie start fine widower new girlfriend child movie filled stupid director choice like let separate coming stupid stupid stupid please waste time hoping get better hardly



Term Frequency – Inverse Document Frequency(TF-IDF)



What the hell is that?

TF-IDF weights words by frequency in one review vs. all reviews .

TF: number of times a word appears in one review

$$\text{IDF}(\text{word}) = \ln\left(\frac{\text{total reviews}}{\text{reviews containing that word}}\right)$$

Eg: "terrible" scores high . "watch" scores low

Lexical representation – captures which words appear, not the meaning behind the word

Output: sparse matrix (10,000 features per review)



Most and Least Sentiment-Discriminative Words (Chi-squared)

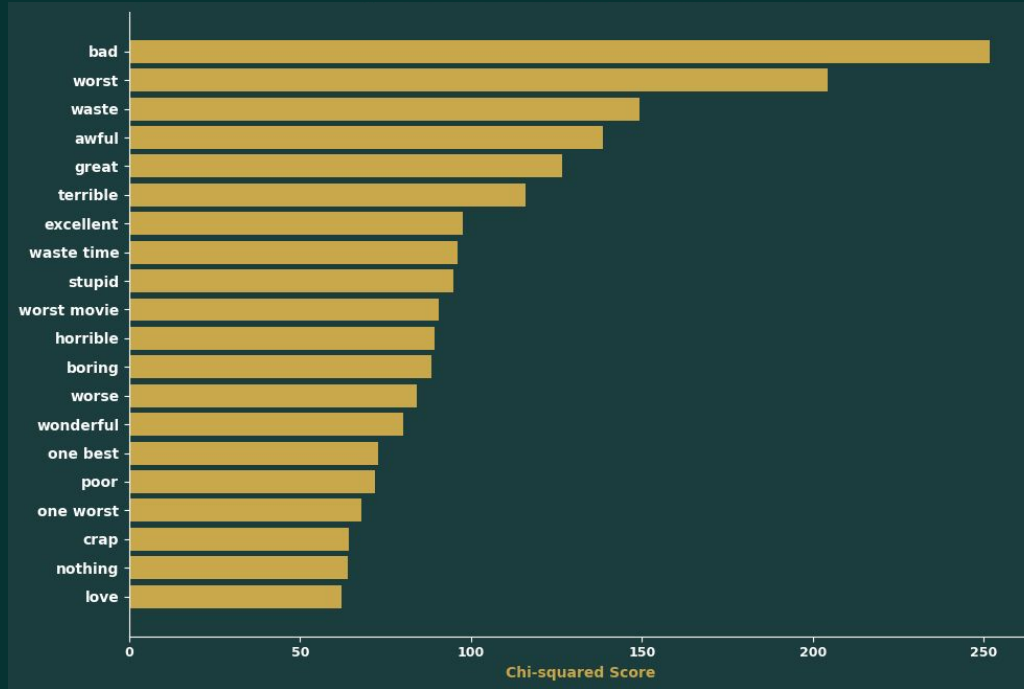
TF-IDF tells us word importance within a review. Chi-squared tells us which words actually matter for predicting sentiment across all 50,000 reviews



Word	TF (total count)	IDF (log)	TF-IDF Score	Chi-squared Score
bad	14496	2.4564	35607.9744	251.6463
worst	4254	3.4239	14565.2706	204.2268
waste	2293	3.9589	9077.7577	149.1379
awful	2590	3.9146	10138.814	138.6327
great	14338	2.3881	34240.5778	126.5907
terrible	2632	3.9314	10347.4448	116.0465
excellent	3257	3.6529	11897.4953	97.5248
waste time	1191	4.5623	5433.6993	96.1195
stupid	2753	3.9216	10796.1648	94.7211
worst movie	933	4.8224	4499.2992	90.3823
...				
leading	947	4.8202	4564.7294	0.0
pulled	417	5.6152	2341.5384	0.0
exotic	157	6.5728	1031.9296	0.0
volume	158	6.6479	1050.3682	0.0
urge	192	6.3496	1219.1232	0.0
carrying	248	6.1119	1515.7512	0.0
like watch	77	7.2529	558.4733	0.0
fatal	175	6.4907	1135.8725	0.0
drug dealer	82	7.24	593.68	0.0
rogers	202	6.6694	1347.2188	0.0



Top 20 Sentiment-Discriminative Words



Latent Dirichlet Allocation(LDA)



★ **Now what's this new thing?** ★

Unsupervised method!! no sentiment labels used, purely finds hidden structure in text

Core assumption: every review is a mixture of topics, every topic is a distribution over words

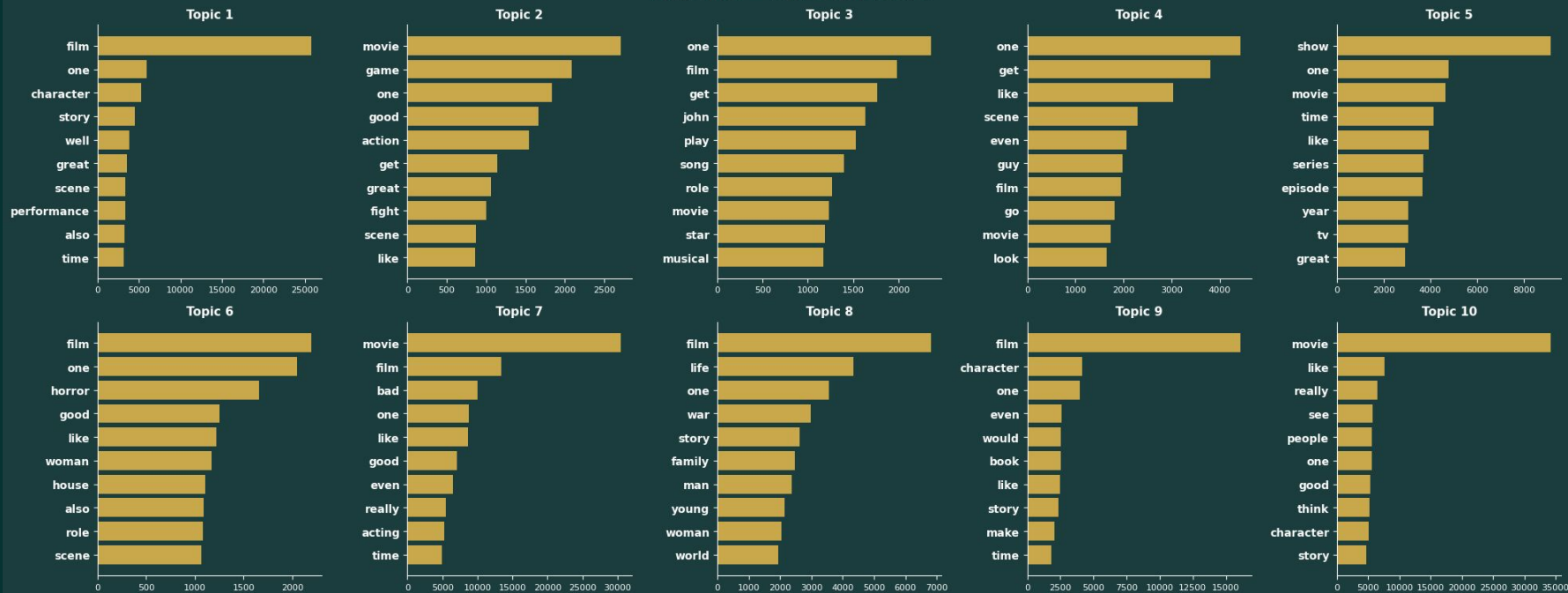
TF (for LDA) = raw word counts (LDA needs actual counts to estimate word probabilities)

Initially tried 15 topics – too noisy, individual words dominated clusters → reduced to 10 for cleaner, more interpretable results ★

★ **Output:** dense vector of 10 numbers per review – topic proportions that sum to 1. ★

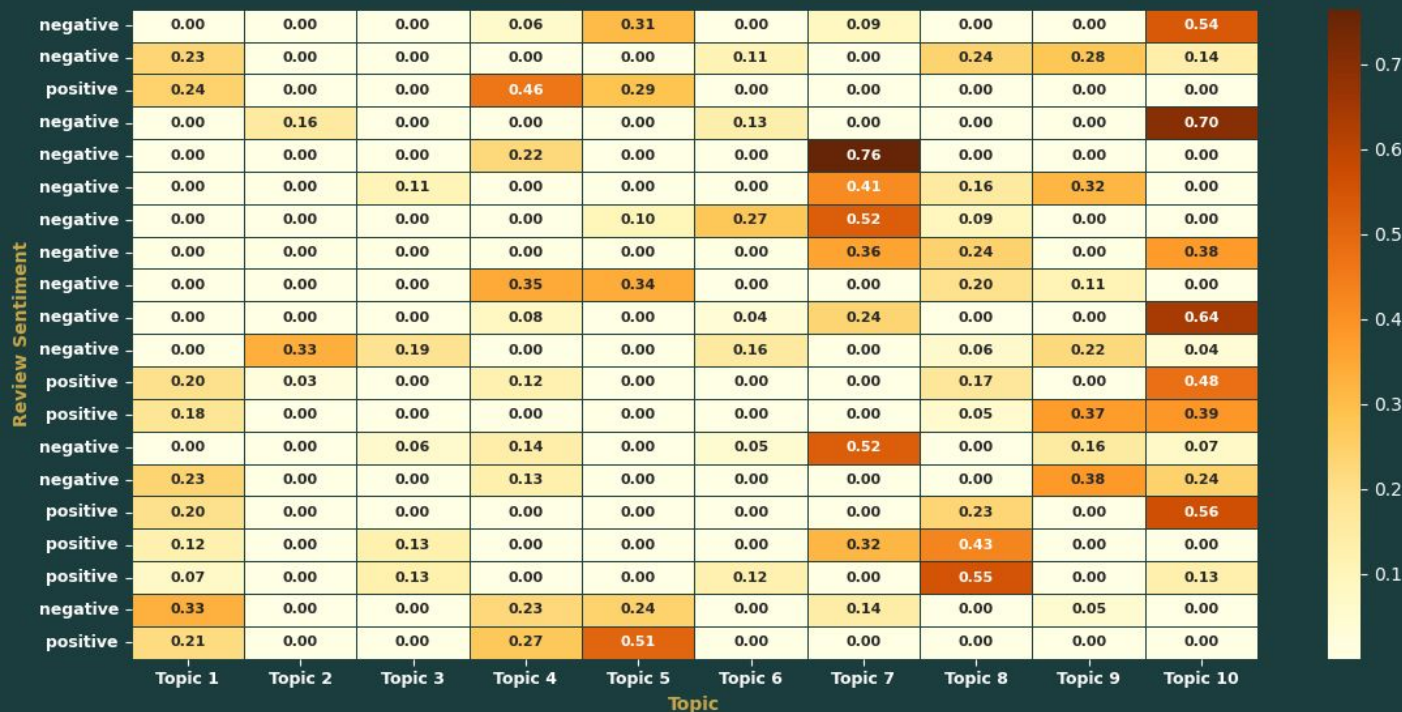
Topic-Word Distribution for LDA

LDA Topics — Top 10 Words Each

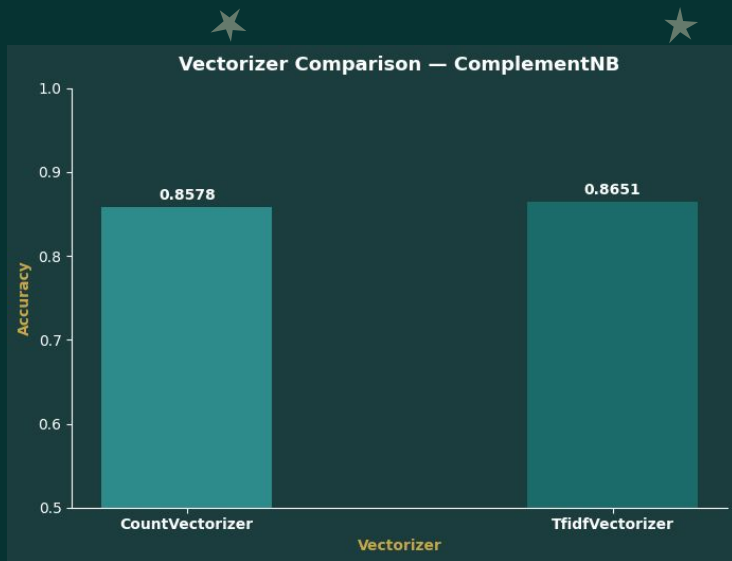


Review-Topic Distribution for LDA

Sample of 20 Reviews



All things vectorizers!!



Converts raw text into numbers.

CountVectorizer: Counting how many times a word appears in a review (int).

TF-IDFVectorizer: Does the same thing but adds weights from tf-idf scores (float).



Feature Engineering

We tested three feature sets:

TF-IDF matrix (10,000 features): from TfidfVectorizer

LDA topic proportions (10 features): from CountVectorizer
LDA

Combined: TF-IDF + LDA stacked horizontally (10,010 features)

*Vectorizers fit only on training set to prevent data leakage
80/20 stratified train/test split · random_state=42 for reproducibility



Classifiers (the good stuff)...

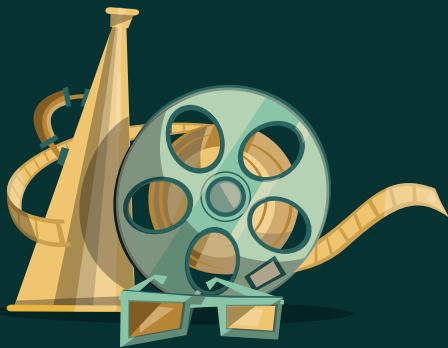
We went with two families of classifiers –

Naive Bayes: probabilistic, assumes feature independence

- ComplementNB + TF-IDF floats
- MultinomialNB + raw integer counts
- GaussianNB + LDA topic proportions(normal dist. assumption)

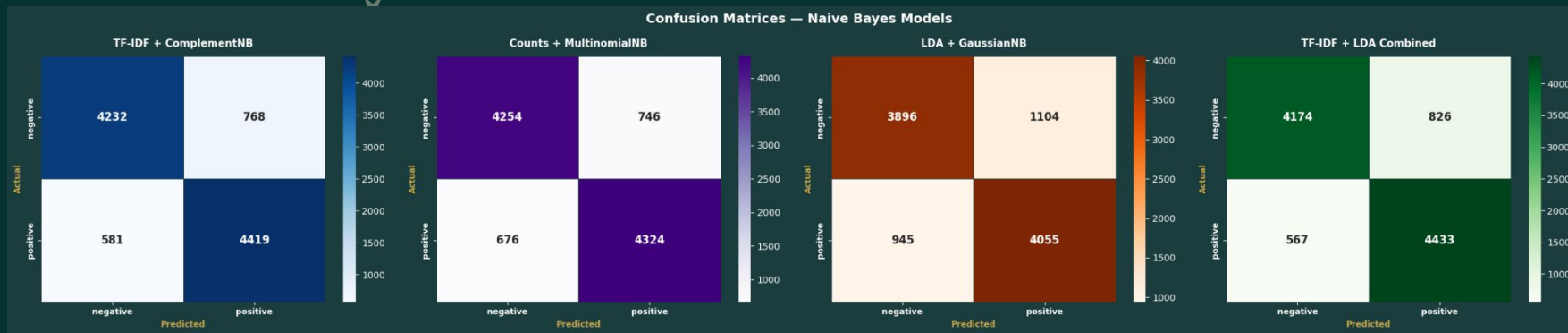
Random Forest: 100 decision trees, majority vote

- No distributional assumptions → works on all three feature sets
- Lets us compare features independently of classifier choice



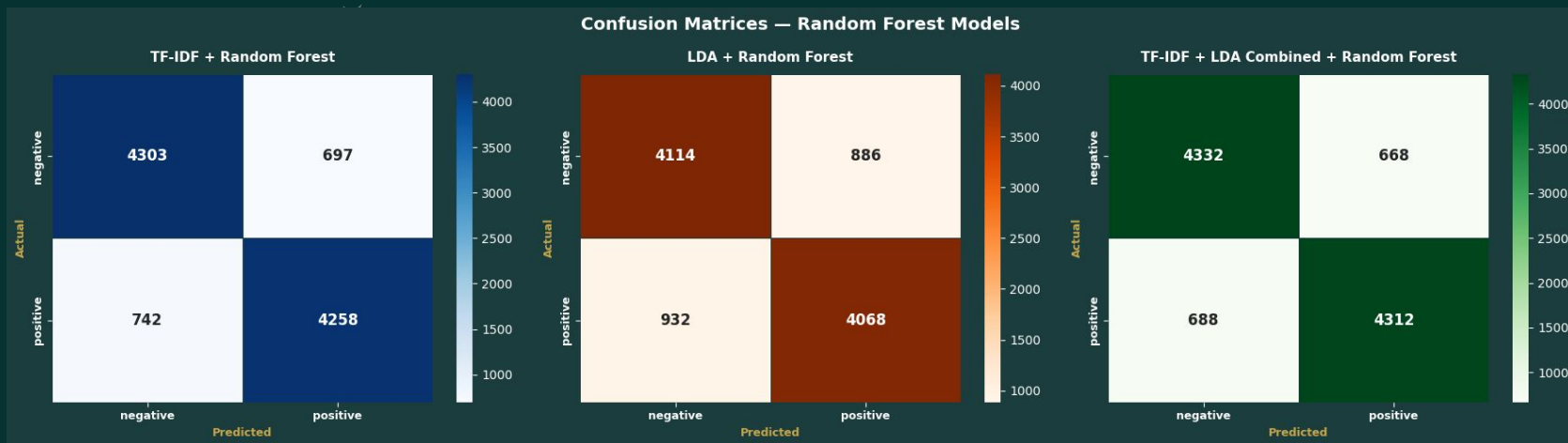
Confusion Matrices for Clarity

For Naive Bayes Family



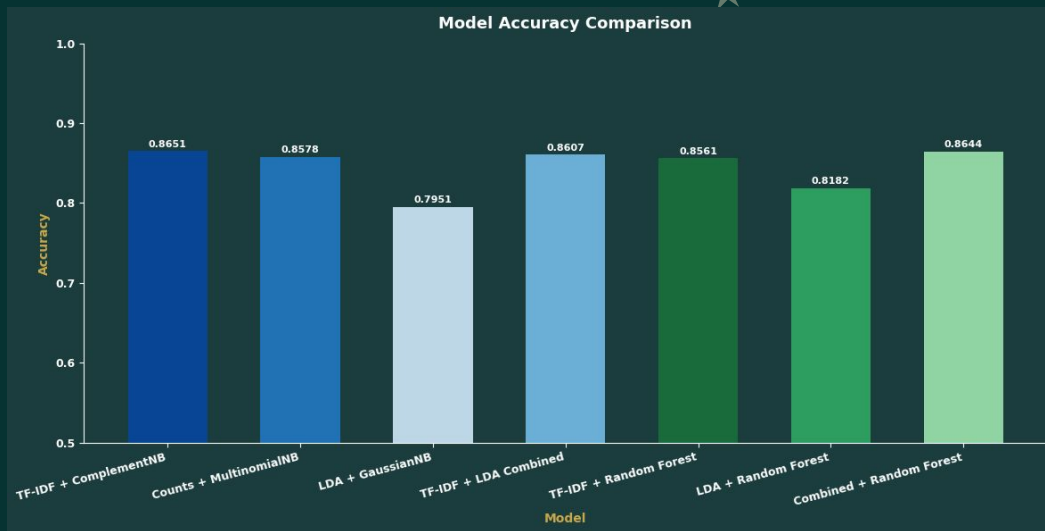
Confusion Matrices for Clarity

For Random Forest



Model Comparison

- **Best overall:** TF-IDF + ComplementNB (86.51%)
Combined + Random Forest is close behind (86.44%), essentially tied.
- Reducing to 10 topics improved LDA alone significantly (79.51% vs 77.35% with 15 topics) – cleaner, less noisy topics.
- LDA alone (79.51%) still the weakest.
- All TF-IDF based models cluster tightly between 85–87% regardless of classifier – suggesting the feature representation matters more than the classifier choice for this task.



Conclusion

- **Simple methods win!!** TF-IDF + ComplementNB (86.51%) — best model overall
- LDA's value is interpretability, not accuracy — topics reveal how people talk about films rather than sentiment
- Random Forest exploits combined features better than Naive Bayes
- 10 topics > 15 topics — less noise, better LDA accuracy (77.35% → 79.51%)
- LDA/QDA classifiers fail at 10,000 dimensions — dimensionality matters



Future Work



★ **Aspect-based sentiment:** not just positive/negative, but which aspect (acting, plot, music)

★ **Hyperparameter tuning:** grid search on Random Forest (n_estimators, max_depth)

★ **Boosting/Bagging:** try XGBoost or AdaBoost on TF-IDF features

★ **Transformers:** BERT/RoBERTa would push accuracy well above 90%

★ **Letterboxd:** apply pipeline to Letterboxd-specific data – more informal, ironic tone

THANK YOU

